

Chapter 26

Automatic Error Analysis

*Given the pace of technology,
I propose we leave math to the machines and go play outside.*

— CALVIN, *Calvin and Hobbes* by Bill Watterson (1992)

*To analyse a given numerical algorithm we proceed as follows.
A number which measures the effect of roundoff error
is assigned to each set of data.
“Hill-climbing” procedures are then applied to search for
values large enough to signal instability.*

— WEBB MILLER, *Software for Roundoff Analysis* (1975)

*The prospects for effective use of interval arithmetic look very good,
so efforts should be made to increase its availability
and to make it as user-friendly as possible.*

— DONALD E. KNUTH, *The Art of Computer Programming,
Volume 2, Seminumerical Algorithms* (1998)

*Despite its wide use,
until quite recently the Nelder-Mead method and its ilk have been
deprecated, scorned, or ignored
by almost all of the mainstream optimization community.*

— MARGARET H. WRIGHT, *Direct Search
Methods: Once Scorned, Now Respectable* (1996)

Automatic error analysis is any process in which we use the computer to help us analyse the accuracy or stability of a numerical computation. The idea of automatic error analysis goes back to the dawn of scientific computing. For example, running error analysis, described in §3.3, is a form of automatic error analysis; it was used extensively by Wilkinson on the ACE machine. Various forms of automatic error analysis have been developed. In this chapter we describe in detail the use of direct search optimization for investigating questions about the stability and accuracy of algorithms. We also describe interval analysis and survey other forms of automatic error analysis.

26.1. Exploiting Direct Search Optimization

Is Algorithm X numerically stable? How large can the growth factor be for Gaussian elimination (GE) with pivoting strategy P? By how much can condition estimator C underestimate the condition number of a matrix? These types of questions are common, as we have seen in this book. Usually, we attempt to answer such questions by a combination of theoretical analysis and numerical experiments with random and nonrandom data. But a third approach can be a valuable supplement to the first two: phrase the question as an optimization problem and apply a direct search method.

A direct search method for the problem

$$\max_{x \in \mathbb{R}^n} f(x), \quad f : \mathbb{R}^n \rightarrow \mathbb{R} \quad (26.1)$$

is a numerical method that attempts to locate a maximizing point using function values only and does not use or approximate derivatives of f . Such methods are usually based on heuristics that do not involve assumptions about the function f . Various direct search methods have been developed; for surveys see Powell [948, 1970], [950, 1998], Swann [1112, 1972], [1113, 1974], Lewis, Torczon, and Trossett [783, 2000], and Wright [1260, 1996]. Most of the methods were developed in the 1960s, in the early years of numerical optimization. For problems in which f is smooth, direct search methods have largely been supplanted by more sophisticated optimization methods that use derivatives (such as quasi-Newton methods and conjugate gradient methods), but they continue to find use in applications where f is not differentiable, or even not continuous. These applications range from chemical analysis [994, 1977], where direct search methods have found considerable use, to the determination of drug doses in the treatment of cancer [105, 1991]; in both applications the evaluation of f is affected by experimental errors. Lack of smoothness of f , and the difficulty of obtaining derivatives when they exist, are characteristic of the optimization problems we consider here.

The use of direct search can be illustrated with the example of the growth factor for GE on $A \in \mathbb{R}^{n \times n}$,

$$\rho_n(A) = \frac{\max_{i,j,k} |a_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|},$$

where the $a_{ij}^{(k)}$ are the intermediate elements generated during the elimination.

We know from §9.3 that the growth factor governs the stability of GE, so for a given pivoting strategy we would like to know how big $\rho_n(A)$ can be.

To obtain an optimization problem of the form (26.1) we let $x = \text{vec}(A) \in \mathbb{R}^{n^2}$, and we define $f(x) := \rho_n(A)$. Then we wish to determine

$$\max_{x \in \mathbb{R}^{n^2}} f(x) \equiv \max_{A \in \mathbb{R}^{n \times n}} \rho_n(A).$$

Suppose, first, that no pivoting is done. Then f is defined and continuous at all points where the elimination does not break down, and it is differentiable except at points where there is a tie for the maximum in the numerator or denominator of the expression defining $\rho_n(A)$. We took $n = 4$ and applied the direct search maximizer MDS (described in §26.2) to $f(x)$, starting with the identity matrix $A = I_4$. After 11 iterations and 433 function evaluations, the maximizer converged¹⁷, having located the matrix¹⁸

$$B = \begin{bmatrix} 1.7846 & -0.2760 & -0.2760 & -0.2760 \\ -3.3848 & 0.7240 & -0.3492 & -0.2760 \\ -0.2760 & -0.2760 & 1.4311 & -0.2760 \\ -0.2760 & -0.2760 & -0.2760 & 0.7240 \end{bmatrix},$$

for which $\rho_4(B) = 1.23 \times 10^5$. (The large growth is a consequence of the submatrix $B(1:3, 1:3)$ being ill conditioned; B itself is well conditioned.) Thus the optimizer readily shows that $\rho_n(A)$ can be very large for GE without pivoting.

Next, consider GE with partial pivoting (GEPP). Because the elimination cannot break down, f is now defined everywhere, but it is usually discontinuous when there is a tie in the choice of pivot element, because then an arbitrarily small change in A can alter the pivot sequence. We applied the maximizer MDS to f , this time starting with the 4×4 instance of the orthogonal matrix¹⁹ A with $a_{ij} = (2/\sqrt{2n+1}) \sin(2ij\pi/(2n+1))$ (cf. (9.12)), for which $\rho_4(A) = 2.32$. After 29 iterations and 1169 function evaluations the maximizer converged to a matrix B with $\rho_4(B) = 5.86$. We used this matrix to start the maximizer AD (described in §26.2); it took 5 iterations and 403 function evaluations to converge to the matrix

$$C = \begin{bmatrix} 0.7248 & 0.7510 & 0.5241 & 0.7510 \\ 0.7317 & 0.1889 & 0.0227 & -0.7510 \\ 0.7298 & -0.3756 & 0.1150 & 0.7511 \\ -0.6993 & -0.7444 & 0.6647 & -0.7500 \end{bmatrix},$$

for which $\rho_4(C) = 7.939$. This is one of the matrices described in Theorem 9.7, modulo the convergence tolerance.

These examples, and others presented below, illustrate the following attractions of using direct search methods to investigate the stability of a numerical computation.

¹⁷In the optimizations of this section we used the convergence tests described in §26.2 with $\text{tol} = 10^{-3}$. There is no guarantee that when convergence is achieved it is to a local maximum; see §26.2.

¹⁸All numbers quoted are rounded to the number of significant figures shown.

¹⁹This matrix is MATLAB's `gallery('orthog', n, 2)`.

(1) The simplest possible formulation of optimization problem is often sufficient to yield useful results. Derivatives are not needed, and direct search methods tend to be insensitive to lack of smoothness in the objective function f . Unboundedness of f is a favourable property—direct search methods usually quickly locate large values of f .

(2) Good progress can often be made from simple starting values, such as an identity matrix. However, prior knowledge of the problem may provide a good starting value that can be substantially improved (as in the partial pivoting example).

(3) Usually it is the global maximum of f in (26.1) that is desired (although it is often sufficient to know that f can exceed a specified value). When a direct search method converges it will, in general, at best have located a *local* maximum—and in practice the maximizer may simply have stagnated, particularly if a slack convergence tolerance is used. However, further progress can often be made by *restarting* the same (or a different) maximizer, as in the partial pivoting example. This is because for methods that employ a simplex (such as the MDS method), the behaviour of the method starting at x_0 is determined not just by x_0 but also by the $n + 1$ vectors in the initial simplex constructed at x_0 .

(4) The numerical information revealed by direct search provides a starting point for further theoretical analysis. For example, the GE experiments above strongly suggest the (well-known) results that $\rho_n(A)$ is unbounded without pivoting and bounded by 2^{n-1} for partial pivoting, and inspection of the numerical data suggests the methods of proof.

When applied to smooth problems the main disadvantages of direct search methods are that they have at best a linear rate of convergence and they are unable to determine the nature of the point at which they terminate (since derivatives are not calculated). These disadvantages are less significant for the problems we consider, where it is not necessary to locate a maximum to high accuracy and objective functions are usually nonsmooth. (Note that these disadvantages are not necessarily shared by methods that implicitly or explicitly *estimate* derivatives using function values, such as methods based on conjugate directions, for which see Powell [948, 1970], [949, 1975]; however, these are not normally regarded as direct search methods.)

A final attraction of direct search is that it can be used to test the correctness of an *implementation* of a stable algorithm. The software in question can be used in its original form and does not have to be translated into some other representation.

26.2. Direct Search Methods

For several years I have been using MATLAB implementations of three direct search methods. The first is the alternating directions (AD) method (also known as the coordinate search method). Given a starting value x it attempts to solve the problem (26.1) by repeatedly maximizing over each coordinate direction in turn:

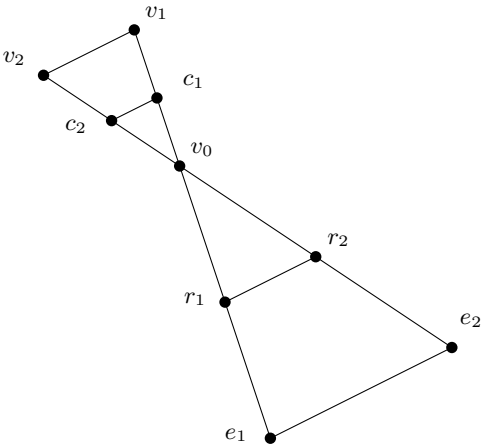


Figure 26.1. *The possible steps in one iteration of the MDS method when $n = 2$.*

```

repeat
    % One iteration comprises a loop over all components of  $x$ .
    for  $i = 1:n$ 
        find  $\alpha$  such that  $f(x + \alpha e_i)$  is maximized (line search)
        set  $x \leftarrow x + \alpha e_i$ 
    end
until converged
    
```

AD is one of the simplest of all optimization methods and the fundamental weakness that it ignores any interactions between the variables is well known. Despite the poor reputation of AD we have found that it can perform well on the types of problems considered here. In our MATLAB implementation of AD the line search is done using a crude scheme that begins by evaluating $f(x + he_i)$ with $h = 10^{-4}x_i$ (or $h = 10^{-4} \max(\|x\|_\infty, 1)$ if $x_i = 0$); if $f(x + he_i) \leq f(x)$ then the sign of h is reversed. Then if $f(x + he_i) > f(x)$, h is doubled at most 25 times until no further increase in f is obtained. Our convergence test checks for a sufficient relative increase in f between one iteration and the next: convergence is declared when

$$f_k - f_{k-1} \leq \text{tol} |f_{k-1}|, \tag{26.2}$$

where f_k is the highest function value at the end of the k th iteration. The AD method has the very modest storage requirement of just a single n -vector.

The second method is the multidirectional search method (MDS) of Dennis and Torczon. This method employs a simplex, which is defined by $n + 1$ vectors $\{v_i\}_0^n$ in $\mathbb{R}^n$. One iteration in the case $n = 2$ is represented pictorially in Figure 26.1, and may be explained as follows.

The initial simplex is $\{v_0, v_1, v_2\}$ and it is assumed that $f(v_0) = \max_i f(v_i)$. The purpose of an iteration is to produce a new simplex at one of whose vertices f exceeds $f(v_0)$. In the first step the vertices v_1 and v_2 are reflected about v_0 along

the lines joining them to v_0 , yielding r_1 and r_2 and the reflected simplex $\{v_0, r_1, r_2\}$. If this reflection step is successful, that is, if $\max_i f(r_i) > f(v_0)$, then the edges from v_0 to r_i are doubled in length to give an expanded simplex $\{v_0, e_1, e_2\}$. The original simplex is then replaced by $\{v_0, e_1, e_2\}$ if $\max_i f(e_i) > \max_i f(r_i)$, and otherwise by $\{v_0, r_1, r_2\}$. If the reflection step is unsuccessful then the edges $v_0 - v_i$ of the original simplex are shrunk to half their length to give the contracted simplex $\{v_0, c_1, c_2\}$. This becomes the new simplex if $\max_i f(c_i) > \max_i f(v_i)$, in which case the current iteration is complete; otherwise the algorithm jumps back to the reflection step, now working with the contracted simplex. For further details of the MDS method see Dennis and Torczon [334, 1991], Kelley [722, 1999, §8.2], and Torczon [1146, 1989], [1147, 1991].

The MDS method requires at least $2n$ independent function evaluations per iteration, which makes it very suitable for parallel implementation. Generalizations of the MDS method that are even more suitable for parallel computation are described in [334, 1991]. The MDS method requires $O(n^2)$ elements of storage for the simplices, but this can be reduced to $O(n)$ (at the cost of extra bookkeeping) if an appropriate choice of initial simplex is made [334, 1991].

Our implementation of the MDS method provides two possible starting simplices, both of which include the starting point x_0 : a regular one (all sides of equal length) and a right-angled one based on the coordinate axes, both as described by Torczon in [1146, 1989]. The scaling is such that each edge of the regular simplex, or each edge of the right-angled simplex that is joined to x_0 , has length $\max(\|x_0\|_\infty, 1)$. Also as in [1146, 1989], the main termination test halts the computation when the relative size of the simplex is no larger than a tolerance tol , that is, when

$$\frac{1}{\max(1, \|v_0\|_1)} \max_{1 \leq i \leq n} \|v_i - v_0\|_1 \leq \text{tol}. \quad (26.3)$$

Unless otherwise stated, we used $\text{tol} = 10^{-3}$ in (26.2) and (26.3) in all our experiments.

The AD and MDS methods are both examples of pattern search methods. Torczon [1148, 1997] shows that for any pattern search method if the level set of f at the starting vector is compact and f is continuously differentiable on a neighborhood of this level set then every limit point of the sequence of iterates is a stationary point (subject to certain technical conditions on the definition of the method). For additional results specific to the MDS method see Torczon [1147, 1991].

The third method that we have used is the Nelder–Mead direct search method [881, 1965], [722, 1999, §8.1], which also employs a simplex but which is fundamentally different from the MDS method. We omit a description since the method is described in textbooks (see, for example, Gill, Murray, and Wright [486, 1981, §4.2.2], or Press, Teukolsky, Vetterling, and Flannery [953, 1992, §10.4]). Our practical experience of the Nelder–Mead method is that while it can sometimes outperform the MDS method, the MDS method is generally superior for our purposes. No convergence results of the form described above for pattern search methods are available for the Nelder–Mead method, and such results cannot exist in view of an example of McKinnon [837, 1998]; see [762, 1998] for what is known.

It is interesting to note that the MDS method, the Nelder–Mead method, and our particular implementation of the AD method do not exploit the numerical values of f : their only use of f is to compare two function values to see which is the larger!

Our MATLAB implementations of the AD, MDS, and Nelder–Mead direct search methods are in the Matrix Computation Toolbox, described in Appendix D.

26.3. Examples of Direct Search

In this section we give examples of the use of direct search to investigate the behaviour of numerical algorithms.

26.3.1. Condition Estimation

MATLAB's function `rcond` uses the LAPACK condition number estimator, described in Algorithm 15.4, to produce a lower bound $\text{est}(A)$ for $\kappa_1(A)$, where $A \in \mathbb{R}^{n \times n}$. We are interested here in automatically generating counterexamples to `rcond`.

To put the problem in the form of (26.1), we define $x = \text{vec}(A)$ and

$$f(x) = \frac{\kappa_1(A)}{\text{est}(A)} \geq 1.$$

We note that, since the algorithm underlying `rcond` contains tests and branches, there are matrices A for which an arbitrarily small change in A can completely change the condition estimate; hence f has points of discontinuity.

We applied the MDS maximizer to `rcond` starting at the 5×5 version of the $n \times n$ matrix with $a_{ij} = \cos((i-1)(j-1)\pi/(n-1))$ (this is a Chebyshev–Vandermonde matrix, as used in §22.3.3, and is `gallery('orthog', n, -1)` in MATLAB). With $\text{tol} = 10^{-6}$, after 59 iterations and 3326 function evaluations the maximizer had located the (well-scaled) matrix

$$A = \begin{bmatrix} 1.1377 & 1.1653 & 0.8503 & 3.3805 & 1.1377 \\ 1.1377 & 0.7964 & 0.3155 & -0.5736 & -0.7518 \\ 1.1377 & -0.2186 & -0.8608 & 0.1404 & 1.4912 \\ 1.1377 & -0.7683 & -2.6908 & 1.5626 & -1.1289 \\ 1.1377 & 0.5523 & 0.5631 & -0.8623 & 1.4928 \end{bmatrix},$$

for which

$$\kappa_1(A) = 8.6 \times 10^6, \quad \text{est}(A) = 3.4 \times 10^1, \quad \frac{\kappa_1(A)}{\text{est}(A)} = 2.5 \times 10^6.$$

With relatively little effort on our part (most of the effort was spent experimenting with different starting matrices), the maximizer has discovered an example where the condition estimator fails to achieve its objective of producing an estimate correct to within an order of magnitude. The value of direct search maximization in this context is clear: it can readily demonstrate the fallibility of

a condition estimator—a task that can be extremely difficult to accomplish using theoretical analysis or tests with random matrices. Moreover, the numerical examples obtained from direct search may provide a starting point for the construction of parametrized theoretical ones, or for the improvement of a condition estimation algorithm.

In addition to measuring the quality of a single algorithm, direct search can be used to compare two competing algorithms to investigate whether one algorithm performs uniformly better than the other. We applied the MDS maximizer to the function

$$f(x) = \frac{\text{estC}(A)}{\text{estR}(A)},$$

where $\text{estR}(A)$ is the condition estimate from `rcond` and $\text{estC}(A)$ is the condition estimate from MATLAB's `condest`, which implements a block 1-norm estimator (see §15.4). If $f(x) > 1$ then `condest` has produced a larger lower bound for $\kappa_1(A)$ than `rcond`. Starting with the same 5×5 starting matrix as above, the MDS maximizer with $\text{tol} = 10^{-6}$ produced after 1426 function evaluations a matrix A for which $\text{estC}(A) = \kappa_1(A) = 2.5 \times 10^6$ and $f(x) = 5.0 \times 10^5$. With f defined as $f(x) = \text{estR}(A)/\text{estC}(A)$ we tried several different starting matrices but were unable to drive f higher than 2. This experiment confirms what we would expect: that the block estimator (iterating with $n \times 2$ matrices—the default in `condest`) provides condition estimates that can be much better than those from `rcond` (which iterates with vectors) and are rarely much worse.

26.3.2. Fast Matrix Inversion

We recall Strassen's inversion method from Problem 23.8: for

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \in \mathbb{R}^{n \times n}, \quad A_{ij} \in \mathbb{R}^{m \times m}, \quad n = 2m,$$

it uses the formulae

$$\begin{aligned} P_1 &= A_{11}^{-1}, & P_2 &= A_{21}P_1, \\ P_3 &= P_1A_{12}, & P_4 &= A_{21}P_3, \\ P_5 &= P_4 - A_{22}, & P_6 &= P_5^{-1}, \\ A^{-1} &= \begin{bmatrix} P_1 - P_3P_6P_2 & P_3P_6 \\ P_6P_2 & -P_6 \end{bmatrix}, \end{aligned}$$

where each of the matrix products is formed using Strassen's fast matrix multiplication method. Strassen's inversion method is clearly unstable for general A , because the method breaks down if A_{11} is singular. Indeed Strassen's inversion method has been implemented on a Cray-2 by Bailey and Ferguson [50, 1988] and tested for $n \leq 2048$, and these authors observe empirically that the method has poor numerical stability. Direct search can be used to gain insight into the numerical stability.

With $x = \text{vec}(A) \in \mathbb{R}^{n^2}$, define the stability measure

$$f(x) = \frac{\min\{\|A\hat{X} - I\|_\infty, \|\hat{X}A - I\|_\infty\}}{\|A\|_\infty \|\hat{X}\|_\infty}, \quad (26.4)$$

where $\hat{X}$ is the inverse of A computed using Strassen's inversion method. This definition of f is appropriate because, as shown in Chapter 14, for most conventional matrix inversion methods either the left residual $\hat{X}A - I$ or the right residual $A\hat{X} - I$ is guaranteed to have norm of order $u\|\hat{X}\|\|A\|$. To treat Strassen's inversion method as favourably as possible we use just one level of recursion; thus P_1 and P_6 are computed using GEPP but the multiplications are done with Strassen's method. We applied the MDS maximizer, with $\text{tol} = 10^{-9}$ in (26.3), starting with the 4×4 Vandermonde matrix whose (i, j) element is $((j-1)/3)^{i-1}$. After 34 iterations the maximizer had converged with $f = 0.838$, which represents complete instability. The corresponding matrix A is well conditioned, with $\kappa_2(A) = 82.4$. For comparison, the value of f when A is inverted using Strassen's method with *conventional* multiplication is $f = 6.90 \times 10^{-2}$; this confirms that the instability is not due to the use of fast multiplication techniques—it is inherent in the inversion formulae.

If A is a symmetric positive definite matrix then its leading principal submatrices are no more ill conditioned than the matrix itself, so we might expect Strassen's inversion method to be stable for such matrices. To investigate this possibility we carried out the same maximization as before, except we enforced positive definiteness as follows: when the maximizer generates a vector $x \equiv \text{vec}(B)$, A in (26.4) is defined as $A = B^TB$. Starting with a 4×4 random matrix A with $\kappa_2(A) = 6.71 \times 10^7$, the maximization yielded the value $f = 3.32 \times 10^{-8}$ after 15 iterations, and the corresponding value of f when conventional multiplication is used is $f = 6.61 \times 10^{-11}$ (the “maximizing” matrix A has condition number $\kappa_2(A) = 3.58 \times 10^9$).

The conclusion from these experiments is that Strassen's inversion method cannot be guaranteed to produce a small left or right residual even when A is symmetric positive definite and conventional multiplication is used. Hence the method must be regarded as being fundamentally unstable.

26.3.3. Roots of a Cubic

Explicit formulae can be obtained for the roots of a cubic equation using techniques associated with the 16th century mathematicians del Ferro, Cardano, Tartaglia, and Vieta [156, 1989], [368, 1990], [882, 1998]. The following development is based on Birkhoff and Mac Lane [114, 1977, §5.5].

Any nondegenerate cubic equation can be put in the form $x^3 + ax^2 + bx + c = 0$ by dividing through by the leading coefficient. We will assume that the coefficients are real. The change of variable $x = y - a/3$ eliminates the quadratic term:

$$y^3 + py + q = 0, \quad p = -\frac{a^2}{3} + b, \quad q = \frac{2}{27}a^3 - \frac{ab}{3} + c.$$

Then Vieta's substitution $y = w - p/(3w)$ yields

$$w^3 - \frac{p^3}{27w^3} + q = 0$$

and hence a quadratic equation in w^3 : $(w^3)^2 + qw^3 - p^3/27 = 0$. Hence

$$w^3 = -\frac{q}{2} \pm \sqrt{\frac{q^2}{4} + \frac{p^3}{27}}. \quad (26.5)$$

For either choice of sign, the three cube roots for w yield the roots of the original cubic, on transforming back from w to y to x .

Are these formulae for solving a cubic numerically stable? Direct search provides an easy way to investigate. The variables are the three coefficients a, b, c and for the objective function we take an approximation to the relative error of the computed roots $\hat{z} \in \mathbb{R}^3$. We compute the “exact” roots z using MATLAB’s `roots` function (which uses the QR algorithm to compute the eigenvalues of the companion matrix²⁰) and then our relative error measure is $\min_{\Pi} \|z - \Pi \hat{z}\|_{\infty} / \|z\|_{\infty}$, where we minimize over all six permutations Π .

First, we arbitrarily take the “+” square root in (26.5). With almost any starting vector of coefficients, the MDS maximizer rapidly identifies coefficients for which the computed roots are very inaccurate. For example, starting with $[1, 1, 1]^T$ we are led to the vector

$$[a \quad b \quad c]^T = [1.732 \quad 1 \quad 1.2704]^T,$$

for which the computed and “exact” roots are

$$\hat{z} = \begin{bmatrix} -1.5999\text{e}+0 \\ -6.6066\text{e}-2 - 8.8557\text{e}-1i \\ -6.6066\text{e}-2 + 8.8557\text{e}-1i \end{bmatrix}, \quad z = \begin{bmatrix} -1.6026\text{e}+0 \\ -6.4678\text{e}-2 + 8.8798\text{e}-1i \\ -6.4678\text{e}-2 - 8.8798\text{e}-1i \end{bmatrix}.$$

When the cubic is evaluated at the computed roots $\hat{z}$ the results are of order 10^{-2} , whereas they are of order 10^{-15} for z . Since the roots are well separated the problem of computing them is not ill conditioned, so we can be sure that z is an accurate approximation to the exact roots. The conclusion is that the formulae, as programmed, are numerically unstable.

Recalling the discussion for a quadratic equation in §1.8, a likely reason for the observed instability is cancellation in (26.5). Instead of always taking the “+” sign, we therefore take

$$w^3 = -\frac{q}{2} - \text{sign}(q) \sqrt{\frac{q^2}{4} + \frac{p^3}{27}}. \quad (26.6)$$

When the argument of the square root is nonnegative, this formula suffers no cancellation in the subtraction; the same is true when the argument of the square root is negative, because then the square root is pure imaginary. With the use of (26.6), we were unable to locate instability using the objective function described above. However, an alternative objective function can be derived as follows. It is reasonable to ask that the computed roots be the exact roots of a slightly perturbed cubic. Thus each computed root $\hat{z}_i$ should be a root of

$$(1 + \Delta d)x^3 + (a + \Delta a)x^2 + (b + \Delta b)x + c + \Delta c = 0,$$

where $\max(|\Delta a|, |\Delta b|, |\Delta c|, |\Delta d|) / \max(|a|, |b|, |c|, 1)$ is of the order of the unit roundoff. Notice that we are allowing the leading coefficient of unity to be perturbed. Denoting the unperturbed cubic by f , we find that this condition implies

²⁰Edelman and Murakami [383, 1995] and Toh and Trefethen [1144, 1994] analyse the stability of this method of finding polynomial roots; the method is stable.

that

$$\max_i \frac{|f(\hat{z}_i)|}{\max(|a|, |b|, |c|, 1) \sum_{j=0}^3 |\hat{z}_i^j|} \quad (26.7)$$

is of order u . We therefore take the quantity in (26.7) as the function to be maximized. On applying the MDS maximizer with starting vector $[1, 1, 1]^T$, we obtain after 10 iterations an objective function value of 1.2×10^{-11} . The cubic coefficient vector is (to three significant figures)

$$[a \quad b \quad c]^T = [-5.89 \times 10^2 \quad 3.15 \times 10^2 \quad -1.36 \times 10^1],$$

and the computed roots are (ignoring tiny, spurious imaginary parts)

$$\hat{z}^T = [4.75 \times 10^{-2} \quad 4.87 \times 10^{-1} \quad 5.89 \times 10^2].$$

The value of the objective function corresponding to the “exact” roots (computed as described above) is of order 10^{-16} (and the value of the previous relative error objective function for the computed roots is of order 10^{-14}).

The conclusion is that even using (26.6) the formulae for the roots are numerically unstable. However, further theoretical analysis is needed to understand the stability fully; see Problem 26.3.

26.4. Interval Analysis

Interval analysis has been an area of extensive research since the 1960s, and it had been considered earlier by Turing and Wilkinson in the 1940s [1243, 1980, p. 104]. As the name suggests, the idea is to perform arithmetic on intervals $[a, b]$ ($b \geq a$). The aim is to provide as the solution to a problem an interval in which the desired result is guaranteed to lie, which may be particularly appropriate if the initial data is uncertain (and hence can be thought of as an interval).

For the elementary operations, arithmetic on intervals is defined by

$$[a, b] \text{ op } [c, d] = \{x \text{ op } y : x \in [a, b], y \in [c, d]\},$$

and the results are given directly by the formulae

$$\begin{aligned} [a, b] + [c, d] &= [a + c, b + d], \\ [a, b] - [c, d] &= [a - d, b - c], \\ [a, b] * [c, d] &= [\min(ac, ab, bc, bd), \max(ac, ab, bc, bd)], \\ [a, b]/[c, d] &= [a, b] * [1/d, 1/c], \quad 0 \notin [c, d]. \end{aligned}$$

We will use the notation $[x]$ for an interval $[x_1, x_2]$ and we define $\text{width}(x) := x_2 - x_1$.

In floating point arithmetic, an interval containing $fl([x] \text{ op } [y])$ is obtained by rounding computations on the left endpoint to $-\infty$ and those on the right endpoint to $+\infty$ (both these rounding modes are supported in IEEE arithmetic).

The success of an interval analysis depends on whether an answer is produced at all (an interval algorithm will break down with division by zero if it attempts

to divide by an interval containing zero), and on the width of the interval answer. A one-line program that prints “ $[-\infty, \infty]$ ” would be correct, robust, and fast, but useless. Interval analysis is controversial because, in general, narrow intervals cannot be guaranteed. One reason is that when dependencies occur in a calculation, in the sense that a variable appears more than once, final interval lengths can be pessimistic. For example, if $[x] = [1, 2]$ then

$$[x] - [x] = [-1, 1], \quad [x]/[x] = [1/2, 2],$$

whereas the optimal intervals for these calculations are, respectively, $[0, 0]$ and $[1, 1]$. These calculations can be interpreted as saying that there is no additive or multiplicative inverse in interval arithmetic. Another source of overestimation is the wrapping effect: the result of an interval computation in 2 or more dimensions is a rectangular box with sides parallel to the coordinate axes, yet it may enclose an image set oriented at an angle to the axes and of much smaller area than the box.

Successful interval computation usually requires algorithms designed for the purpose [1001, 2001]. An example of an algorithm for which interval arithmetic can be ineffective is GEPP, which in general gives very pessimistic error bounds and is unstable in interval arithmetic even for a well-conditioned matrix [893, 1977]. The basic problem is that the interval sizes grow exponentially. For example, in the 2×2 reduction

$$\begin{bmatrix} 1 & [x] \\ 1 & [y] \end{bmatrix} \rightarrow \begin{bmatrix} 1 & [x] \\ 0 & [y] - [x] \end{bmatrix},$$

if $[x] \approx [y]$ then

$$\text{width}([y] - [x]) \approx \text{width}([x] - [x]) = 2 \text{width}([x]).$$

This type of growth is very likely to happen, unlike the superficially similar phenomenon of element growth in standard GEPP. The poor interval bounds are entirely analogous to the pessimistic results returned by early forward error analyses of GEPP (see §9.13). Nickel [893, 1977] states that “The interval Gauss elimination method is one of the most unfavourable cases of interval computation . . . Nearly all other numerical methods give much better results if transformed to interval methods”. Interval GE is effective, however, for certain special classes of matrices, such as M -matrices.

As already mentioned, there is a large body of literature on interval arithmetic, though, as Skeel notes (in an article that advocates interval arithmetic), “elaborate formalisms and abstruse notation make much of the literature impenetrable to all but the most determined outsiders” [1043, 1989]. Despite the inscrutable nature of much of the interval literature, interval arithmetic is becoming more popular, partly because interval arithmetic software is now more widely available (see below). In fact, interval arithmetic is now quite widely used in science and engineering, for example in computational geometry [751, 2001]. A particular application is in computer-assisted proofs of mathematical results; see Frommer [448, 2001] for an excellent survey and Tucker [1163, 2002] for a recent result concerning the Lorenz attractor.

One of the perceived drawbacks of interval arithmetic has been the difficulty of exploiting high-performance computers when implementing it. Rump [998, 1999]

shows that by using a representation of intervals by the midpoint and radius, interval arithmetic can be implemented entirely using BLAS of all three levels, with major benefits for efficiency.

Good sources of information include various conference proceedings and the journal *Computing*. The earliest book is by Moore [872, 1966], whose later book [873, 1979] is one of the best references on the subject. Nickel [893, 1977] gives an easy-to-read summary of research up to the mid 1970s. A more recent reference is Alefeld and Herzberger [11, 1983]. A short, very readable introduction to interval computations is given by Kearfott [719, 1996].

Yohe [1268, 1979] describes a Fortran 66 package for performing interval arithmetic in which machine-specific features are confined to a few modules. It is designed to work with a precompiler for Fortran called Augment [280, 1979], which allows the user to write programs as though Fortran had extended data types—in this case an `INTERVAL` type. A version of the package that allows arbitrary precision interval arithmetic by incorporating Brent's multiple precision arithmetic package (see §27.9) is described in [1269, 1980]. In [1267, 1979], Yohe describes general principles of implementing nonstandard arithmetic in software, with particular reference to interval arithmetic. Kearfott [718, 1996] presents a Fortran 90 module for interval arithmetic.

Kulisch and Miranker have proposed endowing a computer arithmetic with a super-accurate inner product, that is, a facility to compute an exactly rounded inner product for any two vectors of numbers at the working precision [755, 1981], [756, 1983], [757, 1986]. This idea was implemented in the package ACRITH from IBM, which employs interval arithmetic [139, 1985]. Anomalies in early versions of ACRITH are described by Kahan and LeBlanc [705, 1985] and Jansen and Weidner [673, 1986]. For a readable discussion of interval arithmetic and the super-accurate inner product, see Rall [970, 1991]. Cogent arguments against adopting a super-accurate inner product are given by Demmel [312, 1991].

Fortran and Pascal compilers (Fortran SC and Pascal-XSC) and a C++ class library that support a super-accurate inner product and interval arithmetic have been developed jointly by researchers at IBM and the University of Karlsruhe [140, 1987], [737, 1993], [738, 1992]. A toolbox of routines written in Pascal-XSC for solving basic numerical problems and verifying the results is presented in [544, 1993].

Rump [1000, 1999] has produced a MATLAB toolbox INTLAB that permits interval computations to be carried out elegantly and efficiently in MATLAB. Multiple precision interval arithmetic can be done using the MPFR-based package described in §27.9.

A proposal for a set of interval BLAS is contained in an appendix to the BLAS Technical Forum Standard document (see §C.1) [88, 2002].

Finally, we explain why any attempt to compute highly accurate answers using interval arithmetic of a fixed precision (possibly combined with a super-accurate inner product) cannot always succeed. Suppose we have a sequence of problems to solve, where the output of one is the input to the next: $x_{i+1} = f_i(x_i)$, $i = 1:n$, for smooth functions $f_i : \mathbb{R} \rightarrow \mathbb{R}$. Suppose x_1 is known exactly (interval width zero). Working in finite precision interval arithmetic, we obtain an interval $[x_2 - \alpha_2, x_2 + \beta_2]$ containing x_2 . This is used as input to the computation of f_2 .

Even under the favourable assumption of no rounding errors in the evaluation of f_2 , we obtain an interval answer whose width must be of order

$$|f_2(x_2) - f_2(x_2 \pm \epsilon_2)| \gtrsim |f'_2(x_2)|\epsilon_2, \quad \epsilon_2 = \min(|\alpha_2|, |\beta_2|)$$

(the interval could be much bigger than $f_2(x_2) - f_2(x_2 \pm \epsilon_2)$, depending on the algorithm used to evaluate f_2). In other words, the width of the interval containing x_3 is roughly proportional to the condition number of f_2 . When the output of the f_2 computation is fed into f_3 the interval width is multiplied by f'_3 . The width of the final interval containing x_{n+1} is proportional to the product of the condition numbers of all the functions f_i and if there are enough functions, or if they are conditioned badly enough, the final interval will provide no useful information. The only way to avoid such a failure is to use variable precision arithmetic or to reformulate the problem to escape the product of the condition numbers of the f_i .

26.5. Other Work

In this section we outline other work on automatic error analysis.

First, we describe a linearized forward error expression that underlies several of the methods. Suppose that the function f is evaluated at the data $D = [d_1, \dots, d_n]^T$ by an algorithm g using intermediate variables $d_{n+1}, \dots, d_N$, where for simplicity f and all the d_i are assumed to be real scalars. Each d_k ($k > n$) is assumed to be the result of an elementary operation, $+$, $-$, $*$, $/$ or square root; denote by δ_k the associated rounding error. Then, to first order, the computed solution $\hat{d}_N$ satisfies

$$\hat{d}_N - f(d_1, \dots, d_n) = \sum_{k=n+1}^N \frac{\partial g}{\partial d_k}(D, \delta) \delta_k. \quad (26.8)$$

Langlois [766, 2001] presents a method called CENA which computes the linearization (26.8) of the error and adds this quantity to the computed solution in order to correct it (to first order). The derivatives are computed by automatic differentiation, and the rounding errors δ_k are computed exactly (for addition, subtraction, and multiplication) or closely approximated (for division and square root), using techniques such as that in §4.3 for addition. The method provides a bound for the errors in evaluating the sum in (26.8), making use of running error analysis. For a certain class of “linear algorithms” constructively defined in [766, 2001], which includes the substitution algorithm for solving triangular systems, the relation (26.8) is exact and the CENA method provides a corrected solution with a strict error bound.

In the 1970s, Miller and his co-authors developed methods and software for automatically searching for numerical instability in algebraic processes [851, 1975], [854, 1978], [856, 1980]. In [851, 1975] Miller defines a quantity $\sigma(d)$ that bounds, to first order, the sensitivity of an algorithm to perturbations in the data d and in the intermediate quantities that the algorithm generates; $\sigma(d)$ is essentially the result of applying the triangular inequality to (26.8) and using $|\delta_k| \leq u$. He then

defines the forward stability measure $\rho(d) = \sigma(d)/\kappa(d)$, where $\kappa(d)$ is a condition number for the problem under consideration. The algorithms to be analysed are required to contain no loops or conditional branches and are presented to Miller's Fortran software in a special numeric encoding. The software automatically computes the partial derivatives needed to evaluate $\rho(d)$, and attempts to maximize ρ using the method of alternating directions. Miller gives several examples illustrating the scope of his software; he shows, for example, that it can identify the instability of the classical Gram-Schmidt method for orthogonalizing a set of vectors.

In [854, 1978], [855, 1978] Miller and Spooner extend the work in [851, 1975] in several ways. The algorithm to be analysed is expressed in a Fortran-like language that allows for-loops but not logical tests. The definition of ρ is generalized and a method of computing it is developed that involves solving a generalized eigenvalue problem. The book by Miller and Wrathall [856, 1980] gives a thorough development of the work of [854, 1978], including a description of the graph theory techniques used to compute the partial derivatives, and it provides further examples of the use of the software. The potential of Miller and Spooner's software for exposing numerical instability is clearly demonstrated by the case studies in these references, yet the software has apparently not been widely used. This is probably largely due to the inability of the software to analyse algorithms expressed in Fortran, or any other standard language.

A different approach to algorithm analysis is taken by Larson and Sameh [768, 1978], [769, 1980] and implemented in software by Larson, Pasternak, and Wisniewski [767, 1983]. Here, errors are measured in a relative rather than an absolute sense, and the stability is analysed at *fixed data* instead of attempting to maximize instability over all data; however, the analysis is still linearized.

The idea of applying automatic differentiation to a computational graph to obtain estimates for the forward error in an algorithm is found not only in the references cited above, but also in the automatic differentiation literature; see Rall [969, 1981], Iri [666, 1991], and Kubota [752, 1991], for example.

Hull [646, 1979] discusses the possibility of applying program verification techniques to computations that are subject to rounding errors, with the aim of proving a program "correct". Difficulties include deciding what "correct" means and formulating appropriate assertions. Although he reports some progress, Hull concludes that "there is not a great deal to be gained by trying to apply the techniques of program verification to programs for numerical calculations".

Bliss, Brunet, and Gallopoulos [141, 1992] develop Fortran preprocessor tools for implementing the local relative error approach of Larson and Sameh. Their tools also implement a statistical technique of Brunet and Chatelin [169, 1989], [225, 1990] in which the result of every floating point operation is randomly perturbed and statistical methods are used to measure the effect on the output of the algorithm.

Rowan [995, 1990] develops another way to search for numerical instability. For an algorithm with data d he maximizes $S(d) = e(d)/\kappa(d)$ using a direct search maximizer he has developed called the subplex method (which is based on the Nelder-Mead simplex method). Here, $e(d) = y_{\text{acc}} - \hat{y}$ is an approximation to the forward error in the computed solution $\hat{y}$, where y_{acc} is a more accurate estimate

of the true solution than $\hat{y}$, and the condition number $\kappa(d)$ is estimated using finite difference approximations. The quantity $S(d)$ is a lower bound on the backward error of the algorithm at d . Fortran software given in [995, 1990] implements this “functional stability analysis”. The software takes as input two user-supplied Fortran subprograms; one implements the algorithm to be tested in single precision, and the other provides a more accurate solution, typically by executing the same algorithm in double precision. The examples in [995, 1990] show that Rowan’s software is capable of detecting numerical instability in a wide variety of numerical algorithms.

A technique called “significance arithmetic” was studied by Ashenhurst, Metropolis, and others in the 1960s. It involves performing arithmetic on unnormalized numbers in such a way that the number of significant digits in the answers provides estimates of the accuracy of those answers. Significance arithmetic is therefore a form of automatic error analysis. For more details see, for example, Ashenhurst and Metropolis [39, 1965], [844, 1977] and Sterbenz [1062, 1974, §7.2]; Sterbenz explains several drawbacks of the technique.

Stoutemyer [1089, 1977] describes the use of the symbolic manipulation package REDUCE to analyse the propagation of data and rounding errors in numerical algorithms.

The CESTAC (permutation-perturbation) method of La Porte and Vignes [170, 1986], [759, 1974], [1196, 1986] takes a statistical approach. It assumes that rounding errors can be modelled by independent random variables and that the global error in a computation can be well approximated by the sum of the local errors. It runs the algorithm in question 2 or 3 times, making a random perturbation of the last bit of the result of each elementary operation, computes the mean M of the final results, and estimates the number of M ’s correct digits using a Student test. This approach is therefore forward error oriented. Chaitin-Chatelin and her co-workers have developed a backward error oriented method called PRECISE. It makes random perturbations in the original data and examines the effect on the residual and the error divided by the residual, producing statistical estimates of various indicators of backward error, forward error, and conditioning. See Brunet [169, 1989], Chatelin and Brunet [225, 1990], and Chaitin-Chatelin and Frayssé [208, 1996]. PRECISE was designed as a tool to explore the behaviour of numerical algorithms as a function of parameters such as mesh size, time step, nonnormality, and nearness to singularity.

Finally, we note that running error analysis (see §3.3) is a form of automatic error analysis and is an attractive alternative to interval arithmetic as a means of computing a posteriori error bounds for almost any numerical algorithm.

26.6. Notes and References

Sections 26.1–26.3.2 and §26.5 are based on Higham [601, 1993].

Another way to solve a cubic is to use Newton’s method to find a real zero, and then to find the other two zeros by solving a quadratic equation. In a detailed investigation, Kahan [693, 1986] finds this iterative method to be preferable to (sophisticated) use of the explicit formulae. Other useful references on the numerical computation of roots of a cubic are Acton [5, 1996, pp. 29–32], Lanczos [765,

1956, Chap. 1], Press, Teukolsky, Vetterling, and Flannery [953, 1992, §5.6], and Uspensky [1175, 1948].

Problems

26.1. Let $A \in \mathbb{R}^{4 \times 4}$ and let $\hat{Q}_{CGS}$ and $\hat{Q}_{MGS}$ be the computed orthogonal QR factors produced by the classical and modified Gram–Schmidt methods, respectively. Use direct search to maximize the functions

$$\begin{aligned} f_1(A) &= \|\hat{Q}_{CGS}^T \hat{Q}_{CGS} - I\|_2, \\ f_2(A) &= \|\hat{Q}_{MGS}^T \hat{Q}_{MGS} - I\|_2, \\ f_3(A) &= f_1(A)/f_2(A). \end{aligned}$$

In order to keep $\kappa_2(A)$ small, try maximizing $f_i(A) - \theta \max(\kappa_2(A) - \mu, 0)$, where θ is a large positive constant and μ is an upper bound on the acceptable condition numbers.

26.2. It is well known that if $A \in \mathbb{R}^{n \times n}$ is nonsingular and $v^T A^{-1} u \neq -1$ then

$$(A + uv^T)^{-1} = A^{-1} - \frac{A^{-1}uv^T A^{-1}}{1 + v^T A^{-1}u}.$$

This is known as the Sherman–Morrison formula (cf. Problem 13.9). For a history and generalizations see Henderson and Searle [559, 1981]. A natural question is whether this formula provides a stable way to solve a rank-1 perturbed linear system. That is, is the following algorithm stable?

```
% Solve  $Cx = (A + uv^T)x = b$ .
Solve  $Ay = b$  for  $y$ .
Solve  $Az = u$  for  $z$ .
 $x = y - (v^T y)(1 + v^T z)^{-1}z$ 
```

(a) Investigate the stability using direct search. Let both linear systems with coefficient matrix A be solved by GEPP. Take A , u , and v as the data and let the function to be maximized be the normwise backward error $\eta_{C,b}$ in the ∞ -norm.

(b) (RESEARCH PROBLEM) Obtain backward and forward error bounds for the method (for some existing analysis see Yip [1266, 1986]).

26.3. (RESEARCH PROBLEM) Investigate the stability of the formulae of §26.3.3 for computing the roots of a cubic.

26.4. (RESEARCH PROBLEM) Use direct search to try to make progress on some of the research problems in Chapters 9, 10, 11, and 15.